ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO HRI

BÀI THỰC HÀNH 06

GIAO TIẾP PHI NGÔN NGỮ ĐIỀU KHIỂN TURTLEBOT 4 BẰNG CỬ CHỈ TAY

Họ và tên sinh viên: Đỗ Việt Anh

Mã sinh viên: 23020719

Lớp: K68E-RE

Giảng viên hướng dẫn: Kỹ sư Nguyễn Quốc Bảo

Mã nguồn GitHub: turtlebot4_bai06

Video demo: Google Drive – Bài 06

Mục lục

1 Mục tiêu và môi trường sử dụng **1**
2 Cấu trúc hệ thống và luồng dữ liệu **2**
3 Nhận diện và quy ước sáu cử chỉ **3**
4 Mô phỏng và chuẩn bị robot **5**
5 Điều khiển hành vi và cơ chế dừng **6**
6 Kết quả kiểm tra tổng hợp **7**
7 Kết quả tiến–lùi bốn chu kỳ **8**
8 Kết quả di chuyển theo quỹ đạo tròn **9**
9 Tạm dừng, hủy và bỏ tay khỏi ảnh **10**
10 Cách chạy và liên kết bài nộp **12**
Tài liệu tham khảo **12**

1 Mục tiêu và môi trường sử dụng

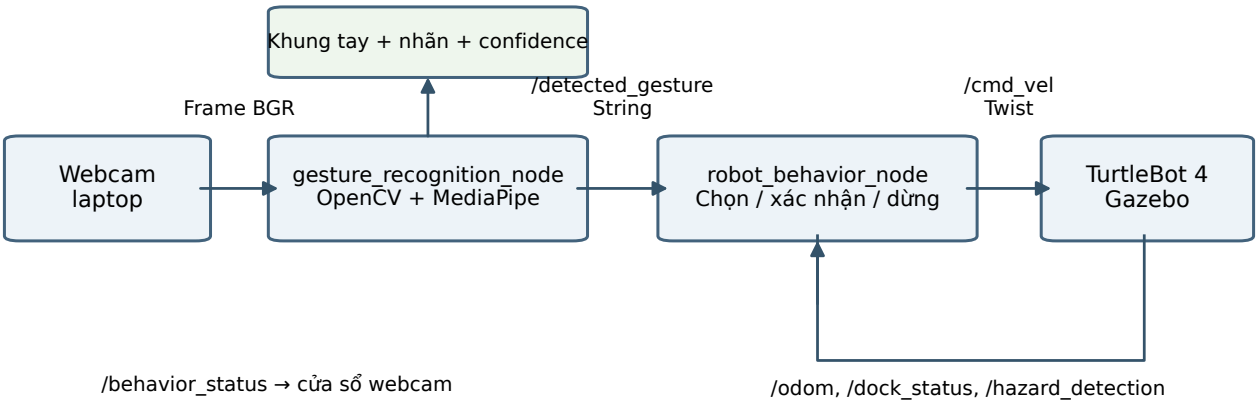
Bài thực hành sử dụng webcam laptop để nhận diện cử chỉ tay bằng OpenCV và MediaPipe, sau đó điều khiển TurtleBot 4 trong Gazebo qua ROS 2. Người điều khiển chọn hành vi bằng tay, xác nhận bằng ngón cái và có thể dừng hoặc hủy khi cần. Các chuyển động trong phần demo được điều khiển bằng cử chỉ.

Thành phần	Môi trường sử dụng
Hệ điều hành, ROS	Ubuntu 22.04, ROS 2 Humble, Python 3.10.12.
Mô phỏng	TurtleBot 4 Standard, Ignition Gazebo Fortress.
Nhận diện	MediaPipe Gesture Recognizer 0.10.21, model `gesture_recognizer.task`.
Ảnh và hiển thị	OpenCV 4.11.0, NumPy 1.26.4, webcam laptop `/dev/video0`.
World	Sàn trống 12 × 12 m; giới hạn điều khiển $\lvert x\rvert, \lvert y\rvert \leq 4{,}5\,\text{m}$.

Bảng 1: *Môi trường triển khai Bài 06.*

`simulation_ready_node` kiểm tra và kích hoạt controller bánh xe rồi gọi undock. Webcam luôn nhận diện tay; dòng trạng thái robot cho biết nếu chưa ra khỏi dock hoặc thiếu odom. File cấu hình mô phỏng đặt chế độ lùi ngay khi khởi động; node chuẩn bị đọc lại cấu hình trước khi cho điều khiển. Không có bước xin quyền lùi bằng cử chỉ hoặc gửi cấu hình sau khi chọn hành vi.

2 Cấu trúc hệ thống và luồng dữ liệu



Hình 1: *Hai node nhận diện và hành vi trao đổi qua topic ROS 2.*

`gesture_recognition_node` dùng OpenCV đọc ảnh, đổi BGR sang RGB và chạy MediaPipe. Node lấy nhãn cùng điểm nhận diện, vẽ 21 điểm và đường nối bàn tay, hiển thị trạng thái robot rồi publish kết quả mới qua `/detected_gesture`.

`robot_behavior_node` subscribe nhãn cử chỉ, lưu lựa chọn và chỉ phát vận tốc sau xác nhận. Timer điều khiển chạy 20 Hz, độc lập với suy luận ảnh để tiếp tục kiểm tra điều kiện dừng khi camera hoặc nhận diện bị gián đoạn.

Topic	Kiểu message	Vai trò
`/detected_gesture`	`std_msgs/msg/String`	Nhãn cử chỉ mới, kể cả `NO_HAND`/`UNKNOWN`.
`/cmd_vel`	`geometry_msgs/msg/Twist`	Vận tốc thẳng `linear.x` và quay `angular.z`.
`/odom`	`nav_msgs/msg/Odometry`	Quãng đường và góc quay thực hiện.
`/dock_status`	`irobot_create_msgs/msg/DockStatus`	Chặn chạy khi robot chưa ra khỏi dock.
`/hazard_detection`	`irobot_create_msgs/msg/HazardDetectionVector`	Nhận bump, cliff, stall, wheel drop.
`/behavior_status`	`std_msgs/msg/String`	JSON trạng thái, hành vi, pha và chu kỳ.

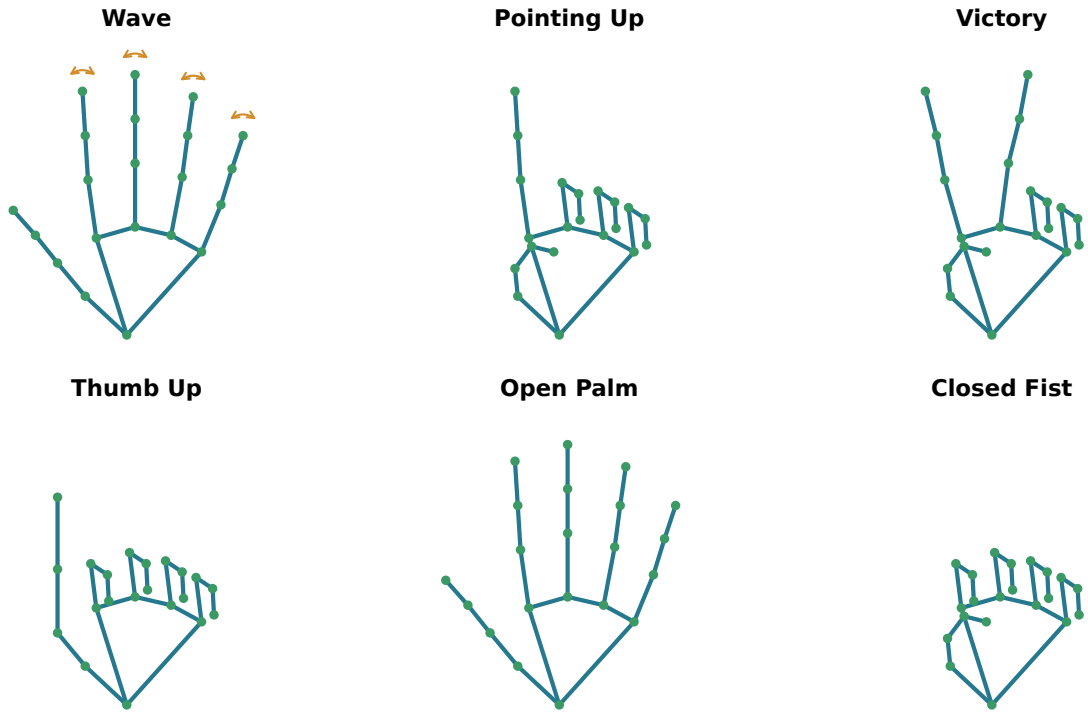
Bảng 2: *Các topic chính.*

Các file trong package. `gesture_recognition_node.py` xử lý webcam và model; `wave_detector.py` nhận chuyển động vẫy tay; `visualization.py` vẽ khung tay. `robot_behavior_node.py` quản lý hành vi và điều kiện dừng. `config/gestures.yaml` lưu ngưỡng nhận diện, tốc độ và thời gian chờ. `launch/demo.launch.py` mở Gazebo, bridge, robot và hai node; `worlds/gesture_arena.sdf` tạo môi trường trống. `gazebo_visualization_node.py` đọc world pose để vẽ quỹ đạo và khung bao; `ign_markers.cpp` gửi marker qua Ignition Transport.

3 Nhận diện và quy ước sáu cử chỉ

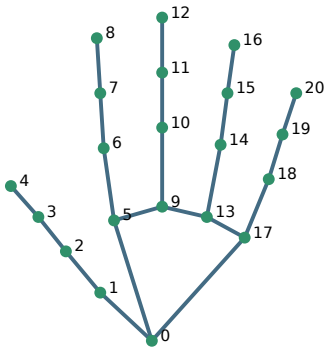
Cử chỉ	Cách thực hiện	Hành vi
Wave	Xòe năm ngón rồi vẫy nhẹ các ngón hai lần.	Chọn chào: xoay trái, phải rồi về hướng gần ban đầu.
Pointing Up	Duỗi ngón trỏ lên, gập các ngón còn lại.	Chọn tiến–lùi 4 chu kỳ, tổng 8 chặng.
Victory	Duỗi ngón trỏ và ngón giữa thành chữ V.	Chọn đi theo quỹ đạo tròn một vòng.
Thumbs Up	Nắm các ngón và giơ ngón cái.	Xác nhận chạy hoặc tiếp tục tác vụ đang PAUSED.
Open Palm	Xòe cả năm ngón về phía camera.	Tạm dừng ngay, giữ hành vi và tiến độ.
Closed Fist	Nắm kín bàn tay.	Hủy hành vi và tiến độ, trở về trạng thái chờ.

Bảng 3: *Quy ước cử chỉ và hành vi. Mọi hành vi chuyển động đều cần xác nhận.*



Hình 2: *Sơ đồ minh họa sáu cử chỉ; Wave có chuyển động các ngón.*

3.1 Landmarks và chuyển động các ngón



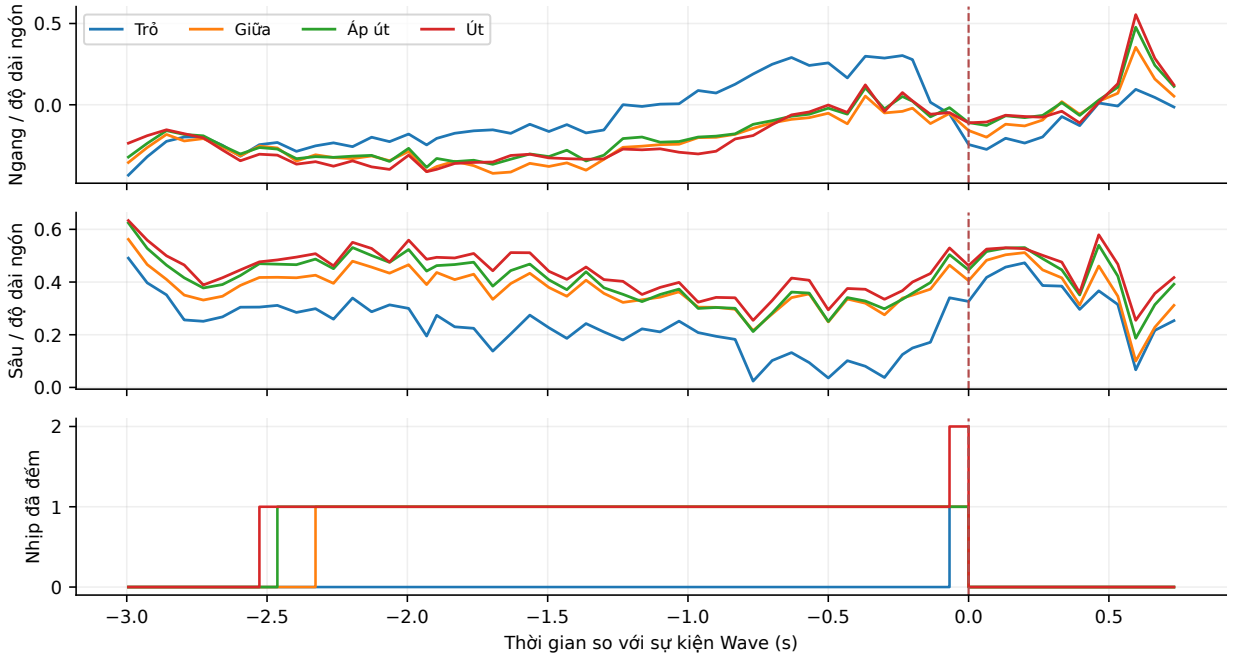
Năm cử chỉ tĩnh. MediaPipe trả về nhãn và confidence. Chương trình dùng ngưỡng chung 0,60. Thumb Up nhận từ 0,55, duy trì từ 0,45 khi ngón cái duỗi và các ngón còn lại co; mỗi frame vẫn cần model dự đoán `Thumb_Up` mới. Confidence là điểm do model trả về, không phải độ chính xác của toàn hệ thống.

Cử chỉ Wave. Xòe năm ngón rồi vẫy nhẹ ít nhất ba ngón hai lần trong 4 s. Chương trình đo đầu ngón trong hệ tọa độ gắn với lòng bàn tay, lọc trung vị ba mẫu và đếm từng ngón riêng. Các ngón có thể lắc hoặc co–duỗi lệch nhịp. Tay mở đứng yên vẫn là Open Palm. Wave là luật theo landmarks, không phải class có sẵn của model.

Hình 3: *Sơ đồ 21 điểm bàn tay.*

Phân biệt Wave và Open Palm. Khi robot đang chạy, Open Palm được chuyển thẳng thành lệnh tạm dừng; không chờ nhận Wave. Lệnh này cần nhãn Open Palm mới đạt ngưỡng từ model; riêng hình dạng landmarks khi model trả UNKNOWN không tạo lệnh dừng. Khi đang IDLE, Wave chỉ tạo lựa chọn chào. Cửa sổ hiển thị bộ đếm 3 s để đổi từ tay mở sang Thumb Up; tay mở trong đoạn này giữ lựa chọn, không tạo lệnh chạy. Hết đoạn chuyển mà vẫn mở tay thì chuyển PAUSED, giữ lựa chọn. UNKNOWN ngắn tối đa 0,18 s không xóa các mẫu xác nhận; nó không xác nhận hành vi. Khi đang thu Wave, nắm giữ 0,4 s mới hủy để phân biệt với pha co ngón; khi đang chạy, nắm tay hủy ngay. Khi đang SELECTED, mất tay hủy lựa chọn.

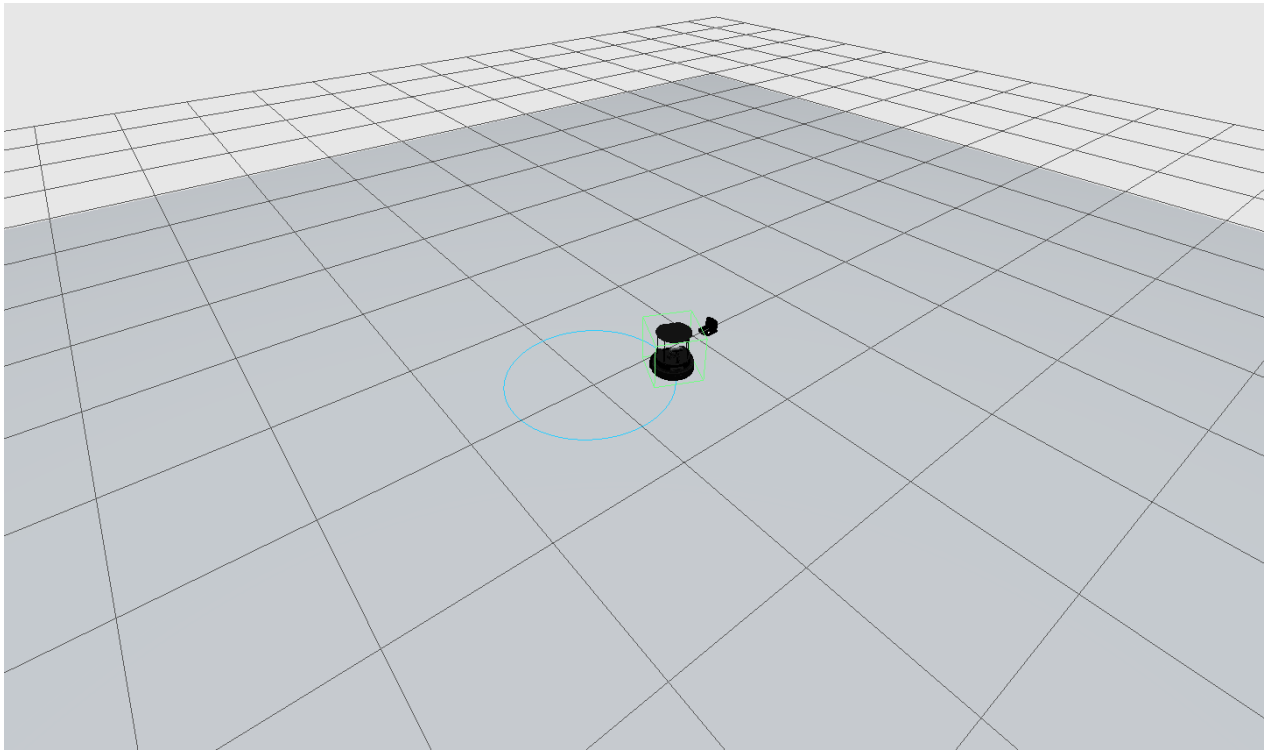
Cửa sổ webcam hiển thị ảnh gương, khung tay, nhãn, điểm model và trạng thái robot. Chỉ một tay được dùng để điều khiển. Kết quả có nhiều tay được coi là mơ hồ và làm robot dừng. Sau xác nhận có thể bỏ tay khỏi ảnh; robot tự chạy hết tác vụ theo odom.



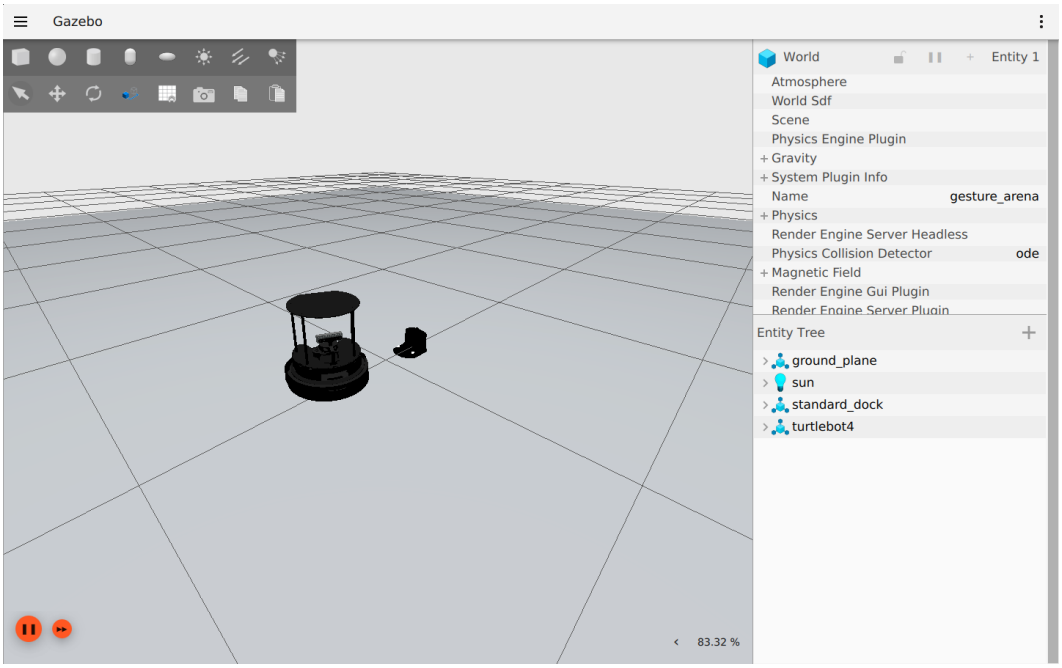
Hình 4: *Đặc trưng đầu ngón từ webcam thật. Vạch đỏ đánh dấu Wave; bộ đếm về 0 sau khi phát sự kiện.*

4 Mô phỏng và chuẩn bị robot

World có sàn trống 12 × 12 m, một TurtleBot 4 Standard và dock. Node chuẩn bị kiểm tra controller bánh xe, cấu hình lùi và thực hiện undock. Hành vi chỉ được xác nhận khi robot đã ra khỏi dock, có odometry mới và không có hazard. `gazebo_visualization_node` vẽ vệt xanh dương từ vị trí robot đã đi và khung bao xanh lá bám theo pose/hướng của mô hình. Khung tham chiếu 0,38 × 0,38 × 0,38 m có lề quanh robot Standard. Tác vụ mới xóa vệt cũ; tạm dừng và tiếp tục giữ nguyên vệt. Marker chỉ nằm trong GUI và không có collision. Lượt kiểm tra đi tròn ghi 251 điểm, gồm một lần PAUSED/resume; vệt không bị xóa khi tiếp tục.

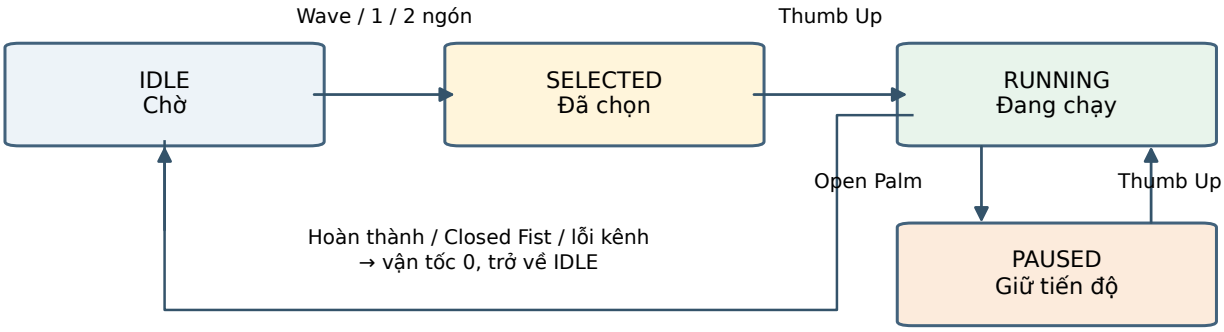


Hình 5: *Vệt quỹ đạo xanh dương và khung bao 3D xanh lá trong Gazebo, lấy từ world pose thật.*



Hình 6: *Cận cảnh mô hình TurtleBot 4 Standard và dock.*

5 Điều khiển hành vi và cơ chế dừng



Hình 7: *Luồng chọn, xác nhận và thực hiện hành vi.*

Chào. Sau Wave rồi Thumb Up, robot quay trái khoảng 0,45 rad, quay phải 0,90 rad, sau đó quay trái 0,45 rad để trở về gần hướng ban đầu. Mỗi pha kết thúc theo góc quay tích lũy từ odometry.

Tiến–lùi. Sau Pointing Up rồi Thumb Up, robot tiến 0,40 m và lùi 0,40 m cho mỗi chu kỳ, lặp bốn chu kỳ. Tiến độ được tính bằng độ dời odom chiếu lên hướng bắt đầu chặng. Giao diện hiển thị số chu kỳ từ 1/4 đến 4/4. Trong Gazebo, launch cấu hình Create 3 ở chế độ `safety_override=backup_only` để cho phép lùi hết 0,40 m; node vẫn kiểm tra hazard, vùng chạy và lệnh dừng.

Đi tròn. Sau Victory rồi Thumb Up, robot xoay tại chỗ để tâm vòng nằm xa dock, rồi chạy với $v = 0{,}12\,\text{m/s}$, $\omega = 0{,}20\,\text{rad/s}$. Bán kính cấu hình là $R = v/\omega = 0{,}60\,\text{m}$. Robot dừng khi tổng góc quay ở pha đi tròn đạt 2π, không chỉ dựa vào thời gian ước lượng.

Điều kiện	Xử lý trong chương trình
Cử chỉ chọn/xác nhận	Cần ít nhất 3 kết quả cùng nhãn, ổn định tối thiểu 0,2 s.
Giữ nguyên cử chỉ	Chỉ tạo một sự kiện; không chạy lại hoặc xếp thêm lệnh trong khi đang chạy.
Open Palm / Thumb Up	Phát Twist bằng 0 ngay, chuyển PAUSED và giữ tiến độ; Thumb Up xác nhận để tiếp tục.
Closed Fist	Hủy cả lựa chọn lẫn tiến độ, phát 0 và trở về IDLE.
NO_HAND / UNKNOWN mới	Sau xác nhận, tác vụ tiếp tục theo odom; ở SELECTED, bỏ tay hủy lựa chọn.
Camera lỗi, nhiều tay, stale	Hủy tác vụ. Kết quả quá 0,25 s bị bỏ; kênh im lặng quá 0,5 s thì watchdog phát 0.
Mất odom hoặc có hazard	Odom quá 0,5 s, ra ngoài vùng chạy hoặc bump/cliff/stall/wheel drop đều dừng.

Bảng 4: *Lọc nhiễu, chống lặp và điều kiện dừng.*

Vận tốc đầu ra bị chặn ở 0,15 m/s và 0,5 rad/s. PAUSED chỉ tiếp tục sau Thumb Up; sau hủy hoặc lỗi kênh, phải chọn và xác nhận lại. Bỏ tay khác với camera bị ngắt. Các khoảng cách, góc và tốc độ trên là giá trị cấu hình điều khiển.

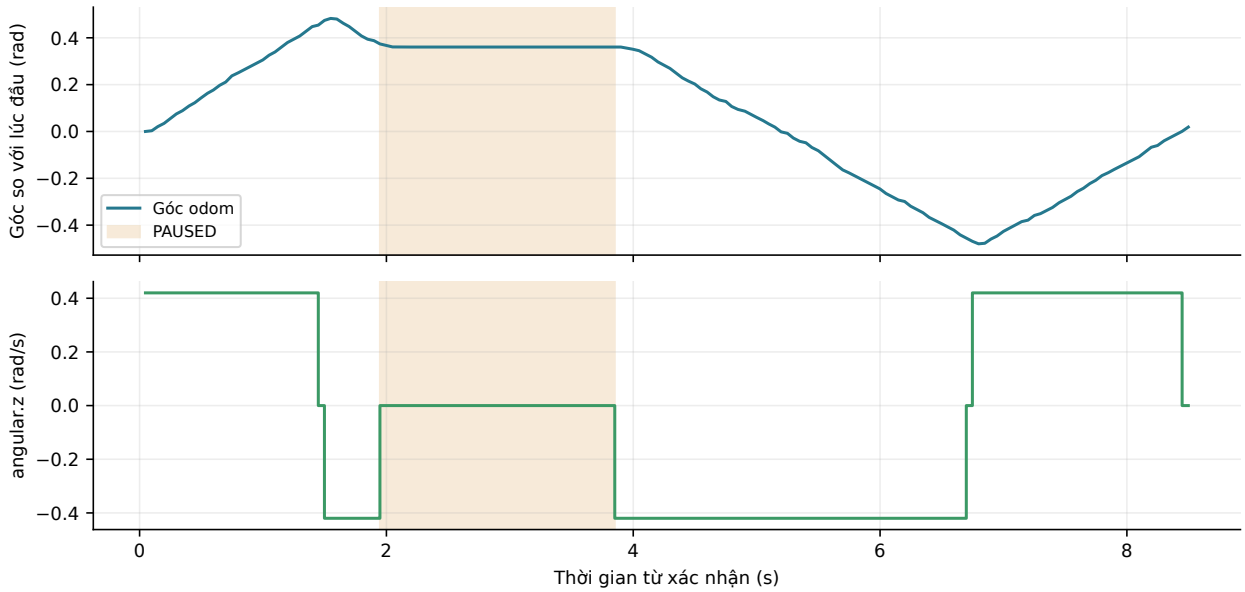
6 Kết quả kiểm tra tổng hợp

Webcam laptop đã đọc được ảnh thật 640 × 480; dữ liệu landmarks được dùng cho hình nhận Wave ở phần trên. Các số liệu chuyển động dưới đây lấy từ Gazebo thật, với nhãn/landmarks thử được phát qua ROS 2. Cách kiểm tra này xác minh node hành vi và cơ chế dừng, chưa thay thế video người thực hiện đủ sáu cử chỉ trên webcam.

Nội dung	Kết quả
Kiểm tra logic	46/46 trường hợp qua; chọn/xác nhận, Wave, Thumb Up, PAUSED/resume, tự chạy và lỗi kênh.
Kiểm tra Gazebo	15/15 nhóm qua; cả ba tác vụ hoàn thành với NO_HAND sau xác nhận và có tạm dừng giữa chừng.
Tiến–lùi	Đủ 4 chu kỳ/8 chặng; độ dài mỗi chặng 0,401–0,411 m. Tạm dừng ở chặng lùi của lượt 3 rồi tiếp tục.
Đi tròn	Góc quay pha CIRCLE đạt 6,292 rad; cách điểm đầu khoảng 0,0113 m.
Tạm dừng	Nhãn Open Palm đến lệnh 0 mất khoảng 0,011–0,036 s. Đây là độ trễ topic, không phải thời gian thân robot dừng hẳn.
Vùng chạy	Không ghi nhận hazard khi RUNNING; tọa độ tuyệt đối lớn nhất 1,545 m, trong giới hạn 4,5 m.

Bảng 5: *Kết quả kiểm tra với đầu vào nhãn thử và cảm biến Gazebo thật.*

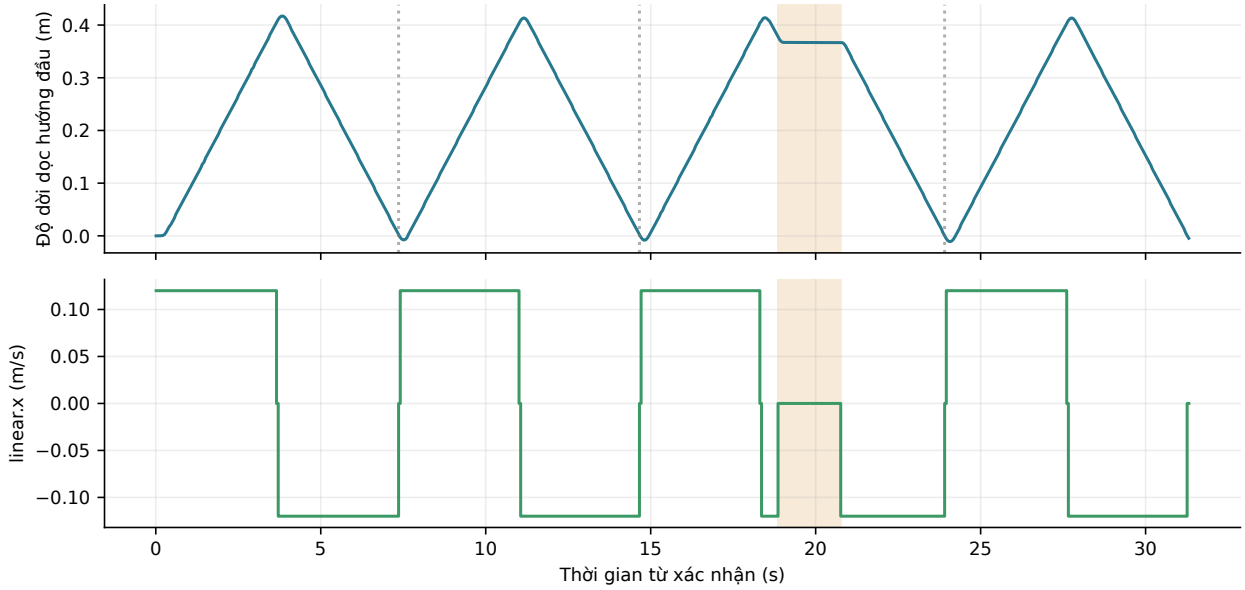
Hành vi chào. Robot thực hiện đủ ba pha trái–phải–về hướng ban đầu. Khoảng tô cam là PAUSED ở pha quay phải; Thumb Up cho tiếp tục pha đó. Sau xác nhận, đầu vào NO_HAND được gửi liên tục đến khi hoàn thành.



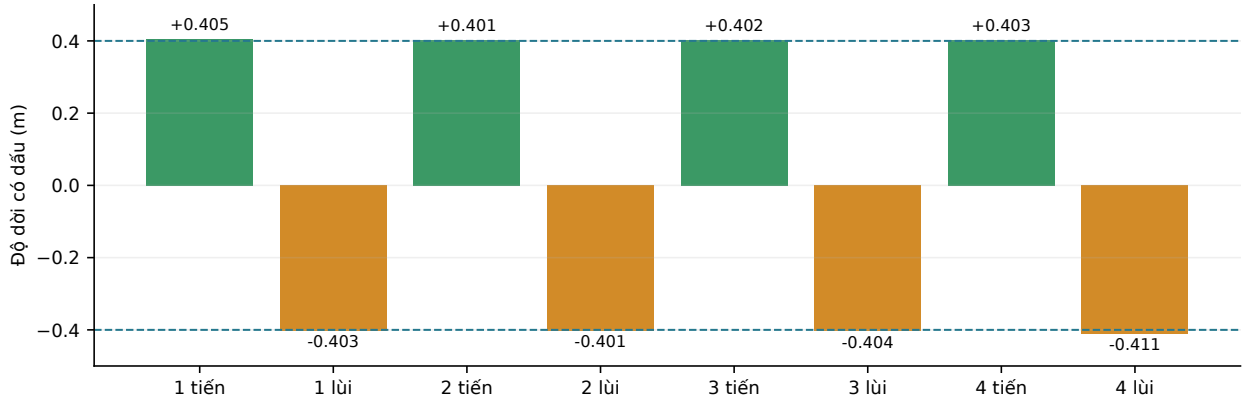
Hình 8: *Góc quay odom và lệnh quay khi chào; khoảng tô cam là tạm dừng.*

7 Kết quả tiến–lùi bốn chu kỳ

Robot tiến rồi lùi bốn lần; mỗi chặng đặt 0,40 m. Đường độ dời đổi chiều đủ tám chặng. Lượt thứ ba có một khoảng tạm dừng ở chặng BACKWARD, sau đó robot tiếp tục phần còn lại và kết thúc lượt thứ tư. Không cần giữ tay trong ảnh sau xác nhận.



Hình 9: *Độ dời odom và lệnh vận tốc của bốn chu kỳ; vùng cam nằm ở lượt 3.*

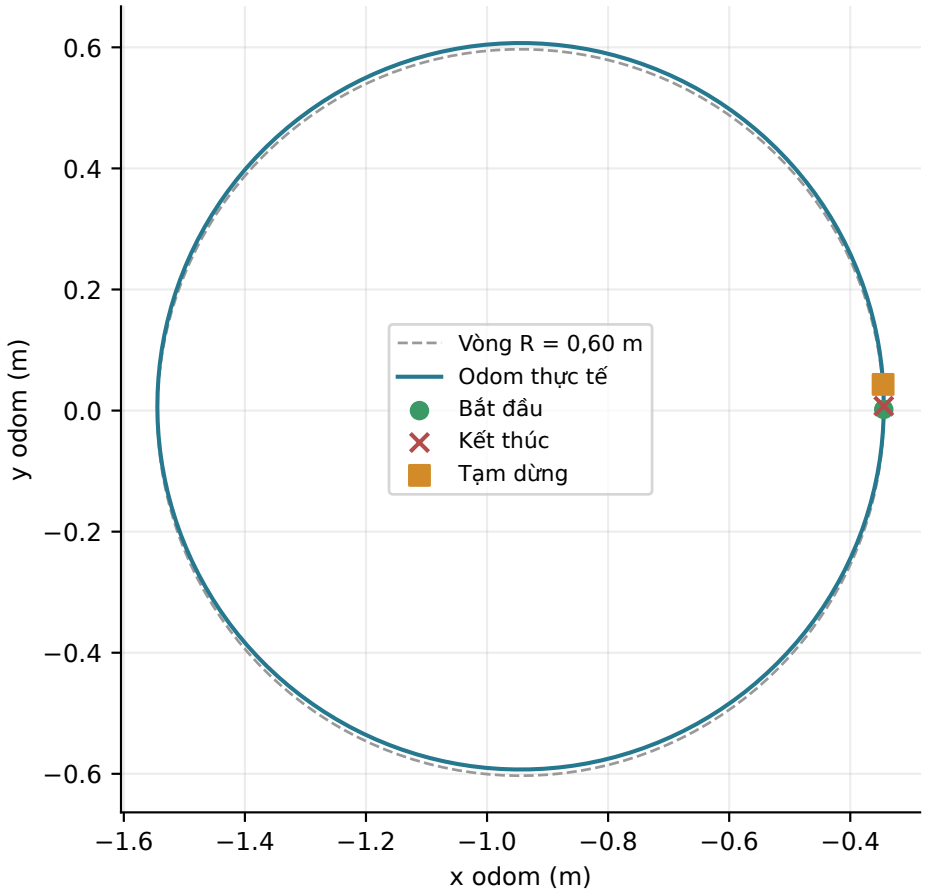


Hình 10: *Tám chặng đo trong Gazebo. Dấu dương là tiến, dấu âm là lùi; đường đứt là mức 0,40 m.*

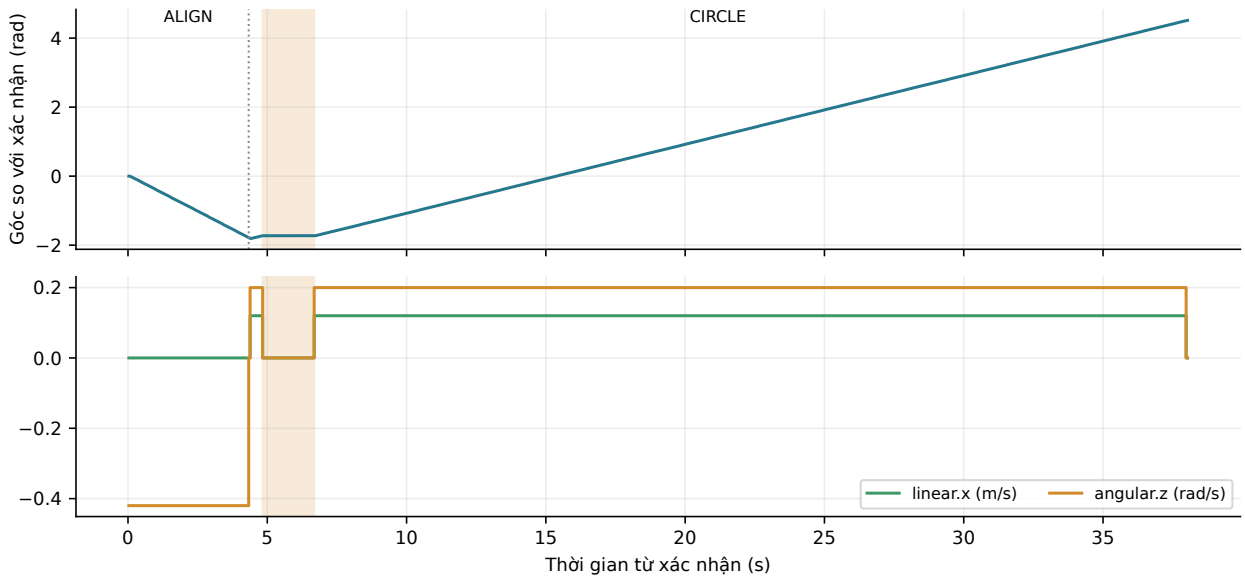
Độ dài nhỏ nhất là 0,401 m và lớn nhất là 0,411 m. Sai lệch gồm bước cập nhật điều khiển và quán tính mô phỏng khi đổi pha; số chu kỳ vẫn được đếm theo odom.

8 Kết quả di chuyển theo quỹ đạo tròn

Sau bước căn hướng, robot dùng vận tốc thẳng 0,12 m/s và vận tốc quay 0,20 rad/s, tương ứng bán kính cấu hình 0,60 m. Quỹ đạo odom tạo thành một vòng; điểm kết thúc cách điểm bắt đầu khoảng 1,13 cm. Ô vuông cam đánh dấu vị trí tạm dừng.



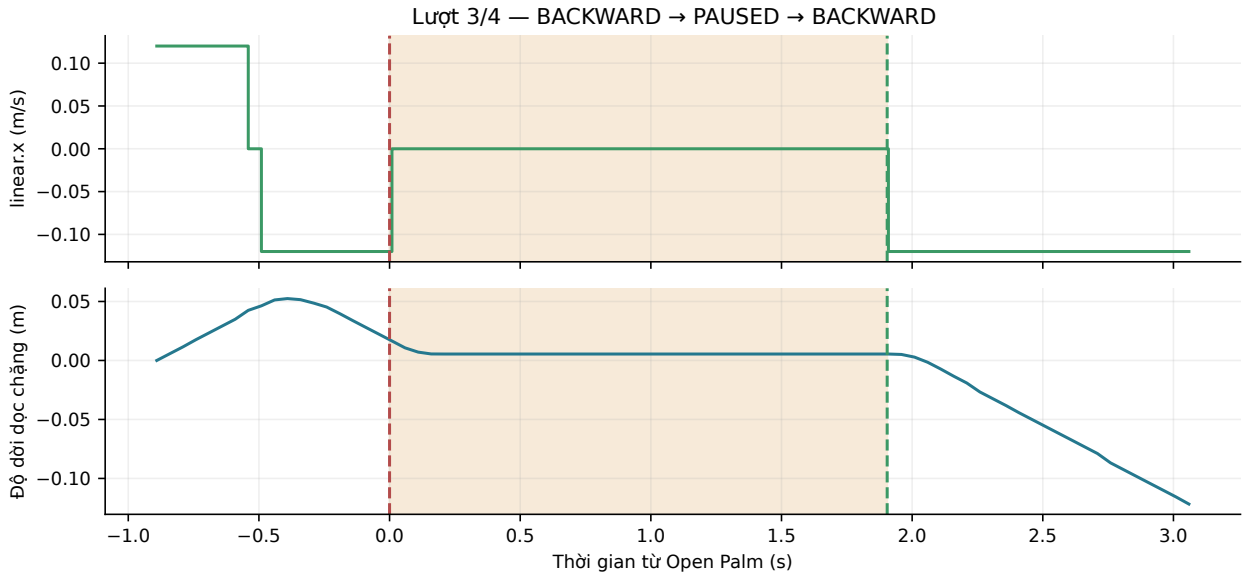
Hình 11: *Quỹ đạo tròn từ odom; đường đứt là vòng tham chiếu bán kính 0,60 m.*



Hình 12: *Góc so với lúc xác nhận, gồm ALIGN rồi CIRCLE; sau PAUSED robot hoàn thành vòng.*

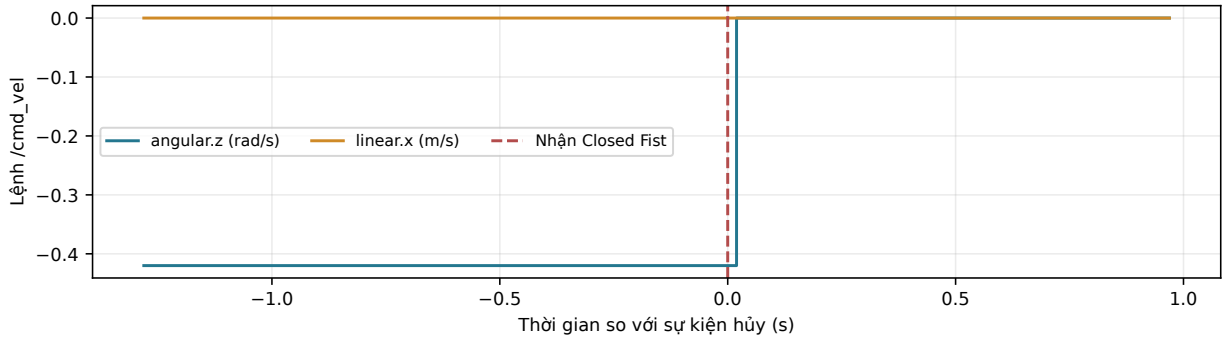
9 Tạm dừng, hủy và bỏ tay khỏi ảnh

Open Palm và Thumb Up. Open Palm phát lệnh 0 ngay trong callback rồi chuyển sang PAUSED. Node giữ hành vi, chặng, chu kỳ và mốc odometry. Ở lượt 3/4 dưới đây, lệnh lùi bằng 0 trong vùng tô cam; độ dời giữ ổn định sau khi robot hết quán tính. Thumb Up cho tiếp tục chặng lùi, không khởi động lại lượt 1.



Hình 13: *Tạm dừng và tiếp tục chặng BACKWARD của lượt 3/4. Vạch đỏ: Open Palm; vạch xanh: Thumb Up.*

Closed Fist. Nắm tay hủy cả lựa chọn lẫn tiến độ và đưa node về IDLE. Thumb Up sau đó không làm robot chạy; cần chọn lại hành vi trước khi xác nhận.

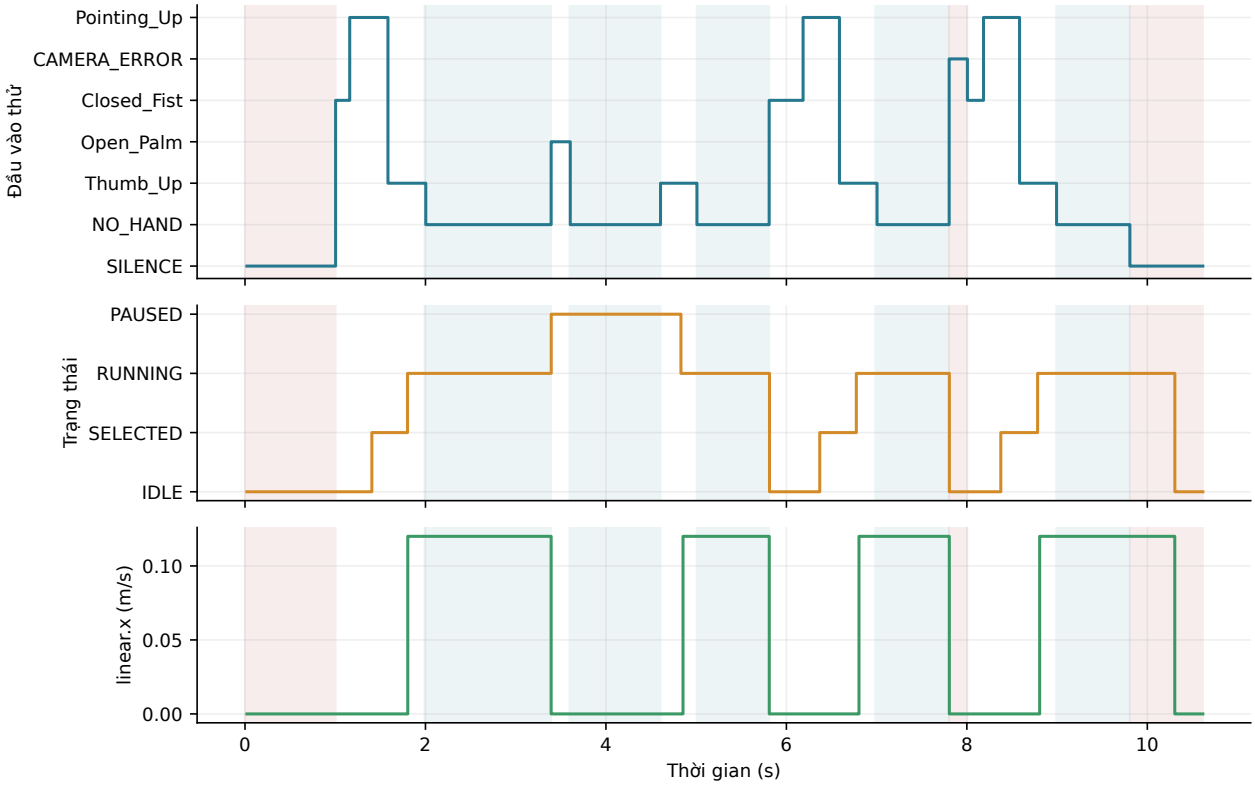


Hình 14: *Lệnh vận tốc về 0 sau Closed Fist trong dữ liệu webcam đã ghi.*

9.1 Phân biệt không có tay và mất kênh camera

Sau xác nhận, robot được phép chạy tự động theo odom. Nếu webcam vẫn gửi kết quả mới NO_HAND hoặc UNKNOWN thì tác vụ tiếp tục. Khi đang PAUSED, bỏ tay không làm robot tự chạy lại. Ở SELECTED, bỏ tay hủy lựa chọn.

Khi camera đọc lỗi/đóng, kết quả quá cũ, phát hiện nhiều tay hoặc node nhận diện ngừng gửi dữ liệu quá 0,5 s, robot dừng và hủy tác vụ. Lỗi đã hết không tự khôi phục chuyển động; người dùng cần chọn rồi xác nhận lại.



Hình 15: *Đầu vào nhãn thử, trạng thái và vận tốc đo qua ROS 2. Nền xanh là NO_HAND; nền đỏ là camera lỗi hoặc im lặng kênh.*

Trong lượt thử này, vận tốc vẫn dương khi đầu vào là NO_HAND. Open Palm đưa về PAUSED, Thumb Up tiếp tục và Closed Fist đưa về IDLE. CAMERA_ERROR làm dừng ngay; đoạn SILENCE cuối chỉ dừng sau khi watchdog hết hạn. Các trạng thái và lệnh trên được ghi từ node thật trong Gazebo.

10 Cách chạy và liên kết bài nộp

Cài đặt. Máy cần ROS 2 Humble, TurtleBot 4 Simulator và Python 3.10. Trong workspace ROS 2, đặt package ở `src/bai06_turtlebot4`. Các bước cài phụ thuộc và build được ghi trong README của repo.

Chạy demo trên máy đã cấu hình.

```
cd /home/admin1/xla/wk4/week6/ros2_ws/src/bai06_turtlebot4
./scripts/run_demo.sh
```

Script mở Gazebo và webcam, căn cửa sổ cạnh nhau, chuẩn bị undock rồi về chờ. Đợi robot ra khỏi dock trước khi xác nhận hành vi. Terminal in thư mục lưu kết quả và ROS domain của phiên chạy. Nhấn Ctrl+C tại terminal đó để đóng phiên.

Thao tác điều khiển. Đưa một tay vào ảnh, thực hiện Wave, Pointing Up hoặc Victory để chọn; khi màn hình hiện SELECTED thì giơ Thumb Up để xác nhận. Sau xác nhận có thể bỏ tay khỏi ảnh. Xòe tay để tạm dừng; giơ Thumb Up để tiếp tục phần còn lại. Nắm tay để hủy và chọn lại từ đầu. Không dùng bàn phím phát vận tốc.

Mã nguồn và video.

- GitHub: github.com/vanhhhhh512/turtlebot4_bai06.
- Video demo: thư mục Google Drive Bài 06.
- Báo cáo PDF và source LaTeX: thư mục `docs/report/` trong repo.

Tài liệu tham khảo

1. Google AI Edge, *Gesture recognition task guide.* `https://developers.google.com/edge/mediapipe/solutions/vision/gesture_recognizer`
2. Google AI Edge, *Gesture recognition guide for Python.* `https://developers.google.com/edge/mediapipe/solutions/vision/gesture_recognizer/python`
3. OpenCV, *Video I/O.* `https://docs.opencv.org/4.x/d8/dfe/classcv_1_1VideoCapture.html`
4. TurtleBot 4, *Simulation.* `https://turtlebot.github.io/turtlebot4-user-manual/software/simulation.html`